```
import numpy as np
import pandas as pd
```

```
df  = pd.read_csv('Pima_Indians_Diabetes_Database.csv')
df.head()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	6	148	72	35	0	33.6	0.627	50	1
1	1	85	66	29	0	26.6	0.351	31	0
2	8	183	64	0	0	23.3	0.672	32	1
3	1	89	66	23	94	28.1	0.167	21	0
4	0	137	40	35	168	43.1	2.288	33	1

Next steps: Generate code with df | View recommended plots | New interactive sheet

```
df.corr()['Outcome']
```

	Outcome
Pregnancies	0.221898
Glucose	0.466581
BloodPressure	0.065068
SkinThickness	0.074752
Insulin	0.130548
BMI	0.292695
DiabetesPedigreeFunction	0.173844
Age	0.238356
Outcome	1.000000

dtype: float64

We observe that BP and skin thickness do not impact outcome much

```
# Let's build a small model
X = df.iloc[:, :-1].values
y = df.iloc[:, -1].values
```

```
X
```

```
array([[  6.   , 148.   ,  72.   , ...,  33.6  ,   0.627,  50.   ],
       [  1.   ,  85.   ,  66.   , ...,  26.6  ,   0.351,  31.   ],
       [  8.   , 183.   ,  64.   , ...,  23.3  ,   0.672,  32.   ],
       ...,
       [  5.   , 121.   ,  72.   , ...,  26.2  ,   0.245,  30.   ],
       [  1.   , 126.   ,  60.   , ...,  30.1  ,   0.349,  47.   ],
       [  1.   ,  93.   ,  70.   , ...,  30.4  ,   0.315,  23.   ]])
```

```
#Since we are building a Neural Network , we need to first scale the input values
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
```

```
X_scaled
```

```
array([[ 0.63994726,  0.84832379,  0.14964075, ...,  0.20401277,
         0.46849198,  1.4259954 ],
       [-0.84488505, -1.12339636, -0.16054575, ..., -0.68442195,
        -0.36506078, -0.19067191],
       [ 1.23388019,  1.94372388, -0.26394125, ..., -1.10325546,
         0.60439732, -0.10558415],
       ...,
       [ 0.3429808 ,  0.00330087,  0.14964075, ..., -0.73518964,
        -0.68519336, -0.27575966],
       [-0.84488505,  0.1597866 , -0.47073225, ..., -0.24020459,
        -0.37110101,  1.17073215],
       [-0.84488505, -0.8730192 ,  0.04624525, ..., -0.20212881,
        -0.47378505, -0.87137393]])
```

```
X.shape
```

```
(768, 8)
```

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size = 0.2, random_state = 42)
```

```
import tensorflow
```

```
from tensorflow import keras
from keras import Sequential
from keras.layers import Dense
```

```
model = Sequential()
model.add(Dense(32, input_dim = 8, activation  = 'relu'))
model.add(Dense(1, activation = 'sigmoid'))
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a laye
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(loss = 'binary_crossentropy', optimizer = 'Adam', metrics = (['accuracy']))
model.fit(X_train, y_train, batch_size = 32, epochs = 100, validation_data = (X_test, y_test))
```

```
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7922 - loss: 0.4125 - val_accuracy: 0.7662 - val_loss: 0.5408
Epoch 73/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8083 - loss: 0.4084 - val_accuracy: 0.7662 - val_loss: 0.5402
Epoch 74/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8073 - loss: 0.3855 - val_accuracy: 0.7662 - val_loss: 0.5403
Epoch 75/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8262 - loss: 0.3880 - val_accuracy: 0.7662 - val_loss: 0.5406
Epoch 76/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7804 - loss: 0.4088 - val_accuracy: 0.7662 - val_loss: 0.5416
Epoch 77/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7864 - loss: 0.4174 - val_accuracy: 0.7662 - val_loss: 0.5410
Epoch 78/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7941 - loss: 0.4020 - val_accuracy: 0.7597 - val_loss: 0.5411
Epoch 79/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8069 - loss: 0.3975 - val_accuracy: 0.7662 - val_loss: 0.5424
Epoch 80/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8254 - loss: 0.3867 - val_accuracy: 0.7727 - val_loss: 0.5419
Epoch 81/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7902 - loss: 0.4009 - val_accuracy: 0.7662 - val_loss: 0.5427
Epoch 82/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8186 - loss: 0.3870 - val_accuracy: 0.7662 - val_loss: 0.5441
Epoch 83/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8108 - loss: 0.3872 - val_accuracy: 0.7662 - val_loss: 0.5450
Epoch 84/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7945 - loss: 0.4050 - val_accuracy: 0.7662 - val_loss: 0.5447
Epoch 85/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7975 - loss: 0.3991 - val_accuracy: 0.7662 - val_loss: 0.5455
Epoch 86/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8250 - loss: 0.3761 - val_accuracy: 0.7662 - val_loss: 0.5455
Epoch 87/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7734 - loss: 0.4186 - val_accuracy: 0.7597 - val_loss: 0.5452
Epoch 88/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8029 - loss: 0.4056 - val_accuracy: 0.7662 - val_loss: 0.5434
Epoch 89/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7985 - loss: 0.4181 - val_accuracy: 0.7727 - val_loss: 0.5430
Epoch 90/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7900 - loss: 0.4174 - val_accuracy: 0.7662 - val_loss: 0.5457
Epoch 91/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7687 - loss: 0.4176 - val_accuracy: 0.7662 - val_loss: 0.5435
Epoch 92/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8078 - loss: 0.3963 - val_accuracy: 0.7727 - val_loss: 0.5417
Epoch 93/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8048 - loss: 0.4116 - val_accuracy: 0.7727 - val_loss: 0.5406
Epoch 94/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8008 - loss: 0.3991 - val_accuracy: 0.7727 - val_loss: 0.5422
Epoch 95/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8061 - loss: 0.3847 - val_accuracy: 0.7727 - val_loss: 0.5421
Epoch 96/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8169 - loss: 0.3958 - val_accuracy: 0.7662 - val_loss: 0.5412
Epoch 97/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8345 - loss: 0.3644 - val_accuracy: 0.7662 - val_loss: 0.5437
Epoch 98/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8028 - loss: 0.3981 - val_accuracy: 0.7597 - val_loss: 0.5435
Epoch 99/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7898 - loss: 0.4248 - val_accuracy: 0.7597 - val_loss: 0.5443
Epoch 100/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7922 - loss: 0.4244 - val_accuracy: 0.7597 - val_loss: 0.5452
<keras.src.callbacks.history.History at 0x7aba1abc7850>
```

Here we need to decide the following hyperparameters ourselves:

1. Number of layers
2. Number of nodes in each layer
3. Activation function
4. Batch Size
5. Number of epochs
6. Loss function
7. Optimizer

It is like trial and error, but can be automated by a library known as **Keras Tuner**

It helps us select the best set of hyperparameters for the dataset for making the model.

```
#1 Selecting the best optimizer
#first install keras tuner
!pip install -U keras-tuner
```

```
Collecting keras-tuner
  Downloading keras_tuner-1.4.7-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: keras in /usr/local/lib/python3.11/dist-packages (from keras-tuner) (3.8.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.11/dist-packages (from keras-tuner) (24.2)
Requirement already satisfied: requests in /usr/local/lib/python3.11/dist-packages (from keras-tuner) (2.32.3)
Collecting kt-legacy (from keras-tuner)
  Downloading kt_legacy-1.0.5-py3-none-any.whl.metadata (221 bytes)
Requirement already satisfied: absl-py in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (1.4.0)
Requirement already satisfied: numpy in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (2.0.2)
Requirement already satisfied: rich in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (0.0.9)
Requirement already satisfied: h5py in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (3.13.0)
Requirement already satisfied: optree in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (0.15.0)
Requirement already satisfied: ml-dtypes in /usr/local/lib/python3.11/dist-packages (from keras->keras-tuner) (0.4.1)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.11/dist-packages (from requests->keras-tuner) (3.4.1)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.11/dist-packages (from requests->keras-tuner) (3.10)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.11/dist-packages (from requests->keras-tuner) (2.4.0)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.11/dist-packages (from requests->keras-tuner) (2025.4.26)
Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/lib/python3.11/dist-packages (from optree->keras->keras-tuner) (4.13.2)
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras->keras-tuner) (3.0.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.11/dist-packages (from rich->keras->keras-tuner) (2.19.1)
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.11/dist-packages (from markdown-it-py>=2.2.0->rich->keras->keras-tuner) (0.1.2)
Downloading keras_tuner-1.4.7-py3-none-any.whl (129 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 129.1/129.1 kB 5.6 MB/s eta 0:00:00
Downloading kt_legacy-1.0.5-py3-none-any.whl (9.6 kB)
Installing collected packages: kt-legacy, keras-tuner
Successfully installed keras-tuner-1.4.7 kt-legacy-1.0.5
```

```
import kerastuner as kt
```

```
<ipython-input-14-5fd8096cdee5>:1: DeprecationWarning: `import kerastuner` is deprecated, please use `import keras_tuner`.
  import kerastuner as kt
```

Steps:

1. Make a function
2. Make a tuner object
3. Pass the function to the tuner object

```
def build_model(hp): #A standard name of the function (build_model),
#hp = hyperparameter is an object of the hyperparameter class which is passed on to the build_model function
  #Build the model here
  model = Sequential()
  model.add(Dense(32, input_dim = 8, activation = 'relu'))
  model.add(Dense(1, activation = 'sigmoid'))
  model.compile(loss = 'binary_crossentropy', optimizer = hp.Choice('optimizer', values = ['adam', 'sgd', 'rmsprop', 'adadelta']), metrics = (['accurac
  # Here, we need to select the optimizer from a list
  # Syntax Signature" optimizer = hp.choice(pass a string say'optimizer', after the comma provide a LIST ['adam', 'sgd', 'rmsprop', 'adadelta'] for 'va
  # This code will build 4 models, each witha different optimizer provided from the list

  return model
```

```
tuner = kt.RandomSearch(build_model, objective = 'val_accuracy', max_trials = 5)
# Try 5 different hyperparameter combinations
# Making an object. The first paramter is the function, 2nd one is the objective, i.e. which thing to reduce/increase, automatically detected
# The tuner will evaluate the combinations using the validation accuracy (val_accuracy) and return the best-performing configuration.
```

```
# Start the hyperparameter search process:
# The tuner will try different combinations of hyperparameters and train the model multiple times.

# X_train, y_train: Training data (features and labels) used to train the model.
# epochs=5: The model will train for 5 epochs (iterations over the training data) for each hyperparameter combination.
# validation_data=(X_test, y_test): Validation data used to evaluate the model performance after each epoch.
# The tuner will use these validation results to determine the best hyperparameter combination.
tuner.search(X_train, y_train, epochs=5, validation_data=(X_test, y_test), verbose=1)
```

```
Trial 4 Complete [00h 00m 02s]
val_accuracy: 0.798701286315918

Best val_accuracy So Far: 0.798701286315918
Total elapsed time: 00h 00m 18s
```

```
tuner.results_summary()
```

```
Results summary
Results in ./untitled_project
Showing 10 best trials
Objective(name="val_accuracy", direction="max")

Trial 3 summary
Hyperparameters:
optimizer: rmsprop
Score: 0.798701286315918

Trial 1 summary
Hyperparameters:
optimizer: adam
Score: 0.7077922224998474
```

```
Trial 2 summary
Hyperparameters:
optimizer: sgd
Score: 0.6883116960525513

Trial 0 summary
Hyperparameters:
optimizer: adadelta
Score: 0.5
```

```
tuner.get_best_hyperparameters()[0].values
```

```
{'optimizer': 'rmsprop'}
```

```
model = tuner.get_best_models(num_models = 1)[0]
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/saving/saving_lib.py:757: UserWarning: Skipping variable loading for optimizer 'rmsprop', because
  saveable.load_own_variables(weights_store.get(inner_path))
```

```
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 32)	288
dense_1 (Dense)	(None, 1)	33

Total params: 321 (1.25 KB)
Trainable params: 321 (1.25 KB)
Non-trainable params: 0 (0.00 B)

```
model.fit(X_train, y_train, batch_size = 32, initial_epoch = 6, epochs = 100, validation_data = (X_test, y_test))
#Since 5 epochs par already trained on best model, so we would like to retain the information
```

```
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 10ms/step - accuracy: 0.7935 - loss: 0.4197 - val_accuracy: 0.7532 - val_loss: 0.5385
Epoch 73/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7947 - loss: 0.4121 - val_accuracy: 0.7532 - val_loss: 0.5401
Epoch 74/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8071 - loss: 0.3905 - val_accuracy: 0.7532 - val_loss: 0.5406
Epoch 75/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8105 - loss: 0.4073 - val_accuracy: 0.7532 - val_loss: 0.5421
Epoch 76/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7828 - loss: 0.4245 - val_accuracy: 0.7532 - val_loss: 0.5431
Epoch 77/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8010 - loss: 0.4081 - val_accuracy: 0.7532 - val_loss: 0.5429
Epoch 78/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7863 - loss: 0.4319 - val_accuracy: 0.7532 - val_loss: 0.5421
Epoch 79/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7962 - loss: 0.4188 - val_accuracy: 0.7532 - val_loss: 0.5423
Epoch 80/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7837 - loss: 0.4374 - val_accuracy: 0.7532 - val_loss: 0.5429
Epoch 81/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8172 - loss: 0.3853 - val_accuracy: 0.7532 - val_loss: 0.5424
Epoch 82/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7713 - loss: 0.4372 - val_accuracy: 0.7532 - val_loss: 0.5430
Epoch 83/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7972 - loss: 0.4196 - val_accuracy: 0.7532 - val_loss: 0.5430
Epoch 84/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8037 - loss: 0.3896 - val_accuracy: 0.7597 - val_loss: 0.5429
Epoch 85/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8220 - loss: 0.3815 - val_accuracy: 0.7468 - val_loss: 0.5450
Epoch 86/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8378 - loss: 0.3687 - val_accuracy: 0.7468 - val_loss: 0.5462
Epoch 87/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8176 - loss: 0.3820 - val_accuracy: 0.7468 - val_loss: 0.5467
Epoch 88/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7867 - loss: 0.4088 - val_accuracy: 0.7532 - val_loss: 0.5450
Epoch 89/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7791 - loss: 0.4250 - val_accuracy: 0.7468 - val_loss: 0.5467
Epoch 90/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8013 - loss: 0.4307 - val_accuracy: 0.7468 - val_loss: 0.5464
Epoch 91/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7790 - loss: 0.4234 - val_accuracy: 0.7468 - val_loss: 0.5469
Epoch 92/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8011 - loss: 0.3965 - val_accuracy: 0.7468 - val_loss: 0.5461
Epoch 93/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8020 - loss: 0.4090 - val_accuracy: 0.7532 - val_loss: 0.5456
Epoch 94/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7750 - loss: 0.4246 - val_accuracy: 0.7468 - val_loss: 0.5475
Epoch 95/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8256 - loss: 0.3810 - val_accuracy: 0.7468 - val_loss: 0.5476
Epoch 96/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7895 - loss: 0.4163 - val_accuracy: 0.7468 - val_loss: 0.5472
Epoch 97/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8077 - loss: 0.4139 - val_accuracy: 0.7532 - val_loss: 0.5471
Epoch 98/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8335 - loss: 0.3675 - val_accuracy: 0.7532 - val_loss: 0.5471
Epoch 99/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8073 - loss: 0.3877 - val_accuracy: 0.7532 - val_loss: 0.5490
Epoch 100/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7848 - loss: 0.4190 - val_accuracy: 0.7532 - val_loss: 0.5484
<keras.src.callbacks.history.History at 0x7ab9b1e826d0>
```

Here accuracy around 75%, SEE val_accuracy

2. Hyperparameter tuning of number of nodes in a given layer

```
def build_model(hp):
  model = Sequential()
  units = hp.Int('units', min_value = 8, max_value = 128, step = 8) # build multiple models, can take any step size
  #First model with 8 neurons in hidden layer, 2nd with 16, till 128 and RETURNS THE MODEL WITH THE BEST RESULTS
  model.add(Dense(units = units, input_dim = 8, activation = 'relu'))
  model.add(Dense(1, activation = 'sigmoid'))
  model.compile(loss = 'binary_crossentropy', optimizer = hp.Choice('optimizer', values = ['adam', 'sgd', 'rmsprop', 'adadelta']), metrics = (['accurac
  return model  #total 16 X 4 = 64 models
```

```
tuner2 = kt.RandomSearch(build_model, objective = 'val_accuracy', max_trials = 5, directory = 'new_dir') # It will pick the already stored model if dir
```

```
tuner2.search(X_train, y_train, epochs = 5, validation_data = (X_test, y_test))
```

```
Trial 5 Complete [00h 00m 03s]
val_accuracy: 0.7857142686843872

Best val_accuracy So Far: 0.7857142686843872
Total elapsed time: 00h 00m 16s
```

```
tuner2.results_summary()
```

```
Results summary
Results in new_dir/untitled_project
Showing 10 best trials
Objective(name="val_accuracy", direction="max")

Trial 0 summary
Hyperparameters:
units: 80
optimizer: adam
Score: 0.7857142686843872

Trial 4 summary
Hyperparameters:
units: 72
optimizer: adam
Score: 0.7857142686843872

Trial 1 summary
Hyperparameters:
units: 120
optimizer: sgd
Score: 0.7402597665786743

Trial 2 summary
Hyperparameters:
units: 24
optimizer: adam
Score: 0.7207792401313782

Trial 3 summary
Hyperparameters:
units: 8
optimizer: adam
Score: 0.6038960814476013
```

```
tuner2.get_best_hyperparameters()[0].values
```

```
{'units': 80, 'optimizer': 'adam'}
```

```
model = tuner2.get_best_models(num_models = 1)[0]
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/saving/saving_lib.py:757: UserWarning: Skipping variable loading for optimizer 'adam', because it
  saveable.load_own_variables(weights_store.get(inner_path))
```

```
model.fit(X_train, y_train, batch_size = 32, initial_epoch = 6, epochs = 100, validation_data = (X_test, y_test))
```

```
Epoch 7/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 1s 16ms/step - accuracy: 0.7757 - loss: 0.5070 - val_accuracy: 0.7792 - val_loss: 0.5101
Epoch 8/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7503 - loss: 0.4898 - val_accuracy: 0.7792 - val_loss: 0.5012
Epoch 9/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.7824 - loss: 0.4633 - val_accuracy: 0.7857 - val_loss: 0.4956
Epoch 10/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7814 - loss: 0.4453 - val_accuracy: 0.7857 - val_loss: 0.4953
Epoch 11/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7996 - loss: 0.4326 - val_accuracy: 0.7727 - val_loss: 0.4964
Epoch 12/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7836 - loss: 0.4440 - val_accuracy: 0.7857 - val_loss: 0.4986
Epoch 13/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8043 - loss: 0.4201 - val_accuracy: 0.7792 - val_loss: 0.5021
Epoch 14/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7743 - loss: 0.4338 - val_accuracy: 0.7727 - val_loss: 0.5046
Epoch 15/100
```

```
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7929 - loss: 0.4431 - val_accuracy: 0.7792 - val_loss: 0.5075
Epoch 16/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7403 - loss: 0.4698 - val_accuracy: 0.7792 - val_loss: 0.5049
Epoch 17/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7879 - loss: 0.4163 - val_accuracy: 0.7727 - val_loss: 0.5093
Epoch 18/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8039 - loss: 0.4104 - val_accuracy: 0.7597 - val_loss: 0.5140
Epoch 19/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8012 - loss: 0.4085 - val_accuracy: 0.7597 - val_loss: 0.5179
Epoch 20/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8211 - loss: 0.4160 - val_accuracy: 0.7597 - val_loss: 0.5164
Epoch 21/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7955 - loss: 0.4185 - val_accuracy: 0.7597 - val_loss: 0.5149
Epoch 22/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7784 - loss: 0.4413 - val_accuracy: 0.7532 - val_loss: 0.5143
Epoch 23/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7660 - loss: 0.4519 - val_accuracy: 0.7597 - val_loss: 0.5164
Epoch 24/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7541 - loss: 0.4446 - val_accuracy: 0.7532 - val_loss: 0.5180
Epoch 25/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7814 - loss: 0.4286 - val_accuracy: 0.7597 - val_loss: 0.5214
Epoch 26/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.7840 - loss: 0.4390 - val_accuracy: 0.7597 - val_loss: 0.5215
Epoch 27/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7946 - loss: 0.4228 - val_accuracy: 0.7597 - val_loss: 0.5224
Epoch 28/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7919 - loss: 0.4291 - val_accuracy: 0.7597 - val_loss: 0.5210
Epoch 29/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7894 - loss: 0.4206 - val_accuracy: 0.7597 - val_loss: 0.5208
Epoch 30/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7998 - loss: 0.4135 - val_accuracy: 0.7597 - val_loss: 0.5211
Epoch 31/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7894 - loss: 0.4282 - val_accuracy: 0.7597 - val_loss: 0.5213
Epoch 32/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7743 - loss: 0.4688 - val_accuracy: 0.7532 - val_loss: 0.5243
Epoch 33/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8248 - loss: 0.3786 - val_accuracy: 0.7532 - val_loss: 0.5253
Epoch 34/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8076 - loss: 0.4149 - val_accuracy: 0.7532 - val_loss: 0.5231
Epoch 35/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.7989 - loss: 0.4100 - val_accuracy: 0.7532 - val_loss: 0.5215
```

Accuracy here = 75%

3. Tuning number of layers

```
def build_model(hp):
  model = Sequential()
  model.add(Dense(72, input_dim = 8, activation = 'relu')) # first hidden layer
  for i in range(hp.Int('num_layers', min_value = 1, max_value = 10)): # it is two loops actually
    model.add(Dense(72, activation = 'relu')) # jitne bhi layers bane by hyperparameter tuning, for each layer 72 nodes, can change it also

  model.add(Dense(1, activation = 'sigmoid'))

  model.compile(loss = 'binary_crossentropy', optimizer = 'rmsprop', metrics = ['accuracy'])

  return model
```

```
tuner3 = kt.RandomSearch(build_model, objective = 'val_accuracy', max_trials = 3, directory = 'new_directory')
```

```
tuner3.search(X_train, y_train, epochs = 5, validation_data = (X_test, y_test))
```

```
Trial 3 Complete [00h 00m 03s]
val_accuracy: 0.7857142686843872

Best val_accuracy So Far: 0.7857142686843872
Total elapsed time: 00h 00m 12s
```

```
tuner3.get_best_hyperparameters()[0].values
```

```
{'num_layers': 4}
```

```
model = tuner3.get_best_models(num_models = 1)[0]
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/saving/saving_lib.py:757: UserWarning: Skipping variable loading for optimizer 'rmsprop', because
  saveable.load_own_variables(weights_store.get(inner_path))
```

```
model.fit(X_train, y_train, batch_size = 32, initial_epoch = 6, epochs = 100, validation_data = (X_test, y_test))
```

```
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9786 - loss: 0.0791 - val_accuracy: 0.7273 - val_loss: 2.4157
Epoch 78/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9988 - loss: 0.0057 - val_accuracy: 0.7273 - val_loss: 2.4982
Epoch 79/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9757 - loss: 0.0509 - val_accuracy: 0.7403 - val_loss: 2.2625
Epoch 80/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9813 - loss: 0.0321 - val_accuracy: 0.7273 - val_loss: 2.4846
Epoch 81/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9878 - loss: 0.0339 - val_accuracy: 0.7273 - val_loss: 2.4488
Epoch 82/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9864 - loss: 0.0350 - val_accuracy: 0.7338 - val_loss: 2.5001
Epoch 83/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9913 - loss: 0.0184 - val_accuracy: 0.7078 - val_loss: 2.5598
Epoch 84/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9997 - loss: 0.0079 - val_accuracy: 0.7143 - val_loss: 2.6009
Epoch 85/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9895 - loss: 0.0290 - val_accuracy: 0.7208 - val_loss: 2.6751
Epoch 86/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9955 - loss: 0.0113 - val_accuracy: 0.6818 - val_loss: 2.8065
Epoch 87/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9787 - loss: 0.0314 - val_accuracy: 0.7143 - val_loss: 2.9211
Epoch 88/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9962 - loss: 0.0177 - val_accuracy: 0.7273 - val_loss: 2.7405
Epoch 89/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9926 - loss: 0.0172 - val_accuracy: 0.7208 - val_loss: 2.9055
Epoch 90/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9903 - loss: 0.0295 - val_accuracy: 0.7273 - val_loss: 2.7747
Epoch 91/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0039 - val_accuracy: 0.7143 - val_loss: 2.9195
Epoch 92/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9856 - loss: 0.0640 - val_accuracy: 0.7143 - val_loss: 2.7776
Epoch 93/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0031 - val_accuracy: 0.7078 - val_loss: 2.9676
Epoch 94/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0015 - val_accuracy: 0.7013 - val_loss: 3.0420
Epoch 95/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9695 - loss: 0.1690 - val_accuracy: 0.7273 - val_loss: 2.9494
Epoch 96/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9912 - loss: 0.0210 - val_accuracy: 0.7208 - val_loss: 3.0264
Epoch 97/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9922 - loss: 0.0156 - val_accuracy: 0.7013 - val_loss: 2.9078
Epoch 98/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 1.0000 - loss: 0.0011 - val_accuracy: 0.7143 - val_loss: 3.0495
Epoch 99/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9758 - loss: 0.0672 - val_accuracy: 0.7208 - val_loss: 3.1288
Epoch 100/100
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.9946 - loss: 0.0083 - val_accuracy: 0.7273 - val_loss: 2.9823
<keras.src.callbacks.history.History at 0x7ab9ab53ce10>
```

Here, Accuracy = 74%

FINAL COMBINATION MODEL

```
from keras.layers import Dropout
def build_model(hp):
    model = Sequential()

    # Number of hidden layers (dynamic based on hyperparameter)
    num_layers = hp.Int('num_layers', min_value=1, max_value=10)

    # Adding layers dynamically
    for i in range(num_layers):
        # If it's the first layer, specify input_dim
        if i == 0:
            model.add(Dense(
                hp.Int('units' + str(i), min_value=8, max_value=128, step=8),  # Units per layer
                activation=hp.Choice('activation' + str(i), values=['relu', 'tanh', 'sigmoid']),  # Activation function
                input_dim=8  # Input dimension for the first layer (assuming 8 features)
            ))
            model.add(Dropout(hp.Choice('dropout' + str(i), values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])))
        else:  # For subsequent layers, no need for input_dim
            model.add(Dense(
                hp.Int('units' + str(i), min_value=8, max_value=128, step=8),  # Units per layer
                activation=hp.Choice('activation' + str(i), values=['relu', 'tanh', 'sigmoid'])  # Activation function
            ))
            model.add(Dropout(hp.Choice('dropout' + str(i), values = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9])))

    # Add output layer
    model.add(Dense(1, activation='sigmoid'))  # Output layer for binary classification

    # Compile the model with dynamic optimizer choice
    model.compile(
        loss='binary_crossentropy',
        optimizer=hp.Choice('optimizer', values=['adam', 'rmsprop', 'sgd', 'nadam', 'adadelta']),
        metrics=['accuracy']
    )

    return model
```

```
tuner_final = kt.RandomSearch(build_model,
                              objective = 'val_accuracy',
                              max_trials = 5,
```

```
                            directory = 'Final_dir',
                            project_name = 'Final_Project')
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a laye
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
tuner_final.search(X_train, y_train, epochs = 5, validation_data = (X_test, y_test))
```

```
Trial 5 Complete [00h 00m 04s]
val_accuracy: 0.6558441519737244

Best val_accuracy So Far: 0.7272727489471436
Total elapsed time: 00h 00m 29s
```

```
tuner_final.get_best_hyperparameters()[0].values
```

```
{'num_layers': 5,
 'units0': 80,
 'activation0': 'relu',
 'dropout0': 0.7,
 'optimizer': 'rmsprop',
 'units1': 88,
 'activation1': 'relu',
 'dropout1': 0.7,
 'units2': 120,
 'activation2': 'tanh',
 'dropout2': 0.7,
 'units3': 16,
 'activation3': 'relu',
 'dropout3': 0.9,
 'units4': 32,
 'activation4': 'relu',
 'dropout4': 0.7,
 'units5': 24,
 'activation5': 'tanh',
 'dropout5': 0.8,
 'units6': 8,
 'activation6': 'tanh',
 'dropout6': 0.8,
 'units7': 64,
 'activation7': 'relu',
 'dropout7': 0.7,
 'units8': 16,
 'activation8': 'tanh',
 'dropout8': 0.6}
```

```
model_final = tuner_final.get_best_models(num_models = 1)[0]
model_final.fit(X_train, y_train, epochs = 200, initial_epoch = 5, validation_data = (X_test, y_test))
```

```
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6433 - loss: 0.6070 - val_accuracy: 0.6429 - val_loss: 0.5710
Epoch 197/200
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6607 - loss: 0.5923 - val_accuracy: 0.6429 - val_loss: 0.5716
Epoch 198/200
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6347 - loss: 0.6000 - val_accuracy: 0.6429 - val_loss: 0.5725
Epoch 199/200
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.6749 - loss: 0.5585 - val_accuracy: 0.6429 - val_loss: 0.5694
Epoch 200/200
20/20 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.6370 - loss: 0.5778 - val_accuracy: 0.6429 - val_loss: 0.5657
<keras.src.callbacks.history.History at 0x7ab9a2f0ef90>
```

```
Start coding or generate with AI.
```